DATA STRUCTURES · DIGITAL NOTES

UNIT IV

Queues and Deques

FIFO structure · Array & linked queues · Circular queue · BFS · Scheduling · Deques

Roadmap of this unit

4.1	Introduction to queues: properties & operations
4.2	Implementing queues using arrays (linear & circular)
4.3	Implementing queues using linked lists
4.4	Priority queue & types of queues
4.5	Applications: BFS, scheduling and more
4.6	Deques: operations, types and applications

4.1 Introduction to Queues

DEFINITION

Queue = a linear data structure in which elements are **inserted at one end (REAR)** and **deleted from the other end (FRONT)**. It follows **FIFO** – **F**irst **I**n, **F**irst **O**ut.

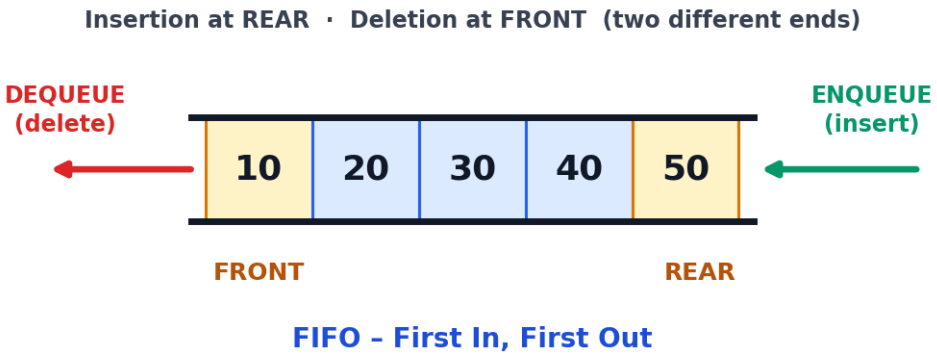


Fig: Queue – enqueue at the rear, dequeue at the front

Real-life examples

- People standing in a **ticket / billing counter line**.
- **Print jobs** waiting for a printer.
- Vehicles at a **toll gate**.
- **Customer-care calls** held in order of arrival.

Properties

- **FIFO** – the earliest arrival is served first (fairness).
- Two pointers: **front** (where we delete) and **rear** (where we insert).
- No random access – only the front element can be removed.
- Every basic operation is **O(1)**.

Operations

Operation	Meaning	Time
enqueue(x)	Insert x at the **rear** (check overflow)	O(1)
dequeue()	Remove and return the **front** element (check underflow)	O(1)
front() / peek()	Read the front element without removing	O(1)
isEmpty()	True if queue has no element	O(1)
isFull()	True if no space (array queue)	O(1)

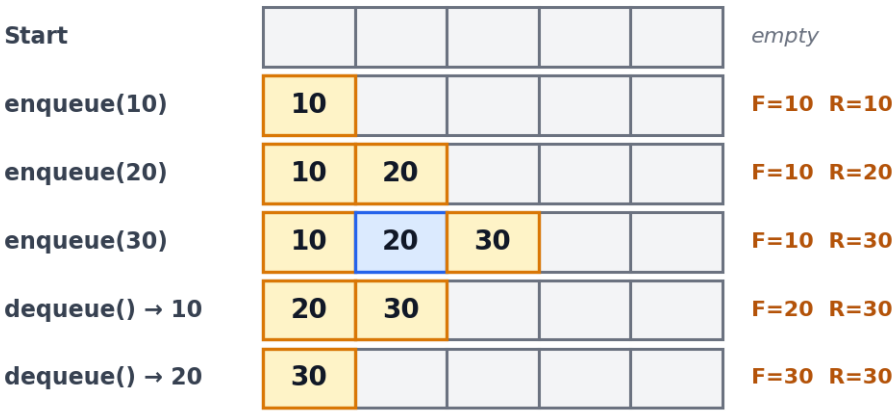


Fig: Queue state after each operation – F = front value, R = rear value

Stack vs Queue

Basis	Stack	Queue
Principle	LIFO	FIFO
Ends used	One end (top)	Two ends (front, rear)
Insert / Delete	push / pop at top	enqueue at rear / dequeue at front
Pointers	top	front and rear
Examples	Undo, recursion, expression evaluation	Scheduling, BFS, buffering

4.2 Queue using an Array

A. Linear (simple) queue

- Array **queue[MAX]**, with **front = −1** and **rear = −1** (empty).
- enqueue: if rear == MAX−1 → overflow; else **rear++**, store (and set front = 0 for the first element).
- dequeue: if front == −1 or front > rear → underflow; else take queue[front] and **front++**.

```
#define MAX 5
int queue[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1) { printf("Queue Overflow\n"); return; }
    if (front == -1) front = 0;            // first element
    queue[++rear] = x;
}
int dequeue() {
    if (front == -1 || front > rear) { printf("Queue Underflow\n"); return -1; }
    return queue[front++];
}
```

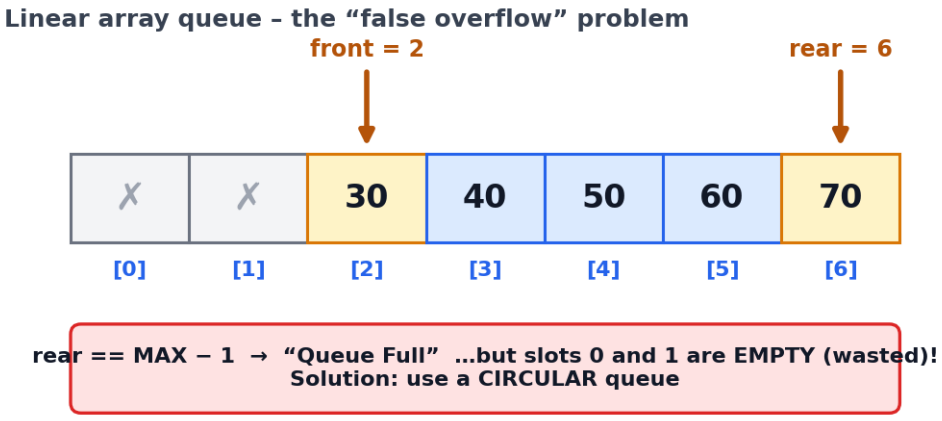


Fig: Problem of the linear queue – freed slots at the front are never reused

B. Circular queue

The array is treated as a **ring**: after the last index the pointers wrap around to index 0. Freed slots are reused, so there is **no false overflow**.

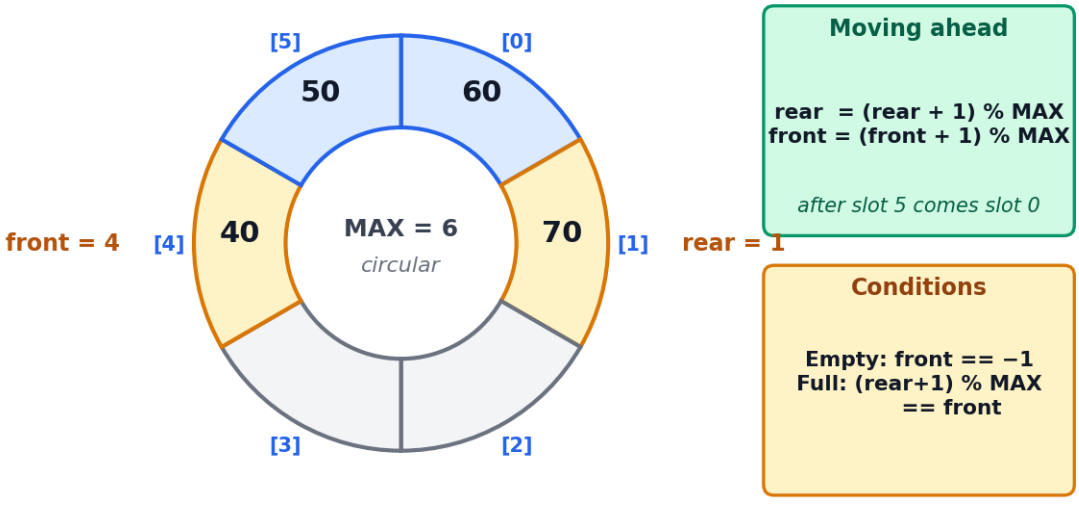


Fig: Circular queue with MAX = 6 – elements 40, 50, 60, 70 wrap around the end of the array

```
#define MAX 5
int cq[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if ((rear + 1) % MAX == front) { printf("Queue Full\n"); return; }
    if (front == -1) front = 0;                 // first insertion
    rear = (rear + 1) % MAX;                    // wrap around
    cq[rear] = x;
}
int dequeue() {
    if (front == -1) { printf("Queue Empty\n"); return -1; }
    int x = cq[front];
    if (front == rear) front = rear = -1;       // last element removed
    else front = (front + 1) % MAX;
    return x;
}
```

A circular queue of size MAX can hold at most MAX elements with this test

Dry run of a circular queue (MAX = 4)

Operation	front	rear	cq[0] [1] [2] [3]	Remark
(start)	−1	−1	_ _ _ _	empty
enqueue(10)	0	0	10 _ _ _	first element
enqueue(20), (30)	0	2	10 20 30 _	
dequeue()	1	2	(10) 20 30 _	returns 10
enqueue(40)	1	3	(10) 20 30 40	rear at last index
enqueue(50)	1	0	50 20 30 40	**wraps to index 0**; now full
enqueue(60)	1	0	50 20 30 40	(0+1)%4 == 1 → **Full**

Basis	Linear queue	Circular queue
Pointer movement	rear++ / front++	(rear+1) % MAX
Memory reuse	No – wasted slots	**Yes**
Full condition	rear == MAX − 1	(rear + 1) % MAX == front
Empty condition	front == −1 or front > rear	front == −1
Efficiency	Poor	Better

EXAM TIP
Remember the formula **(index + 1) % MAX** – it is the heart of every circular-queue question. Dry-run with MAX = 4 to see how the wrap-around works.

4.3 Queue using a Linked List

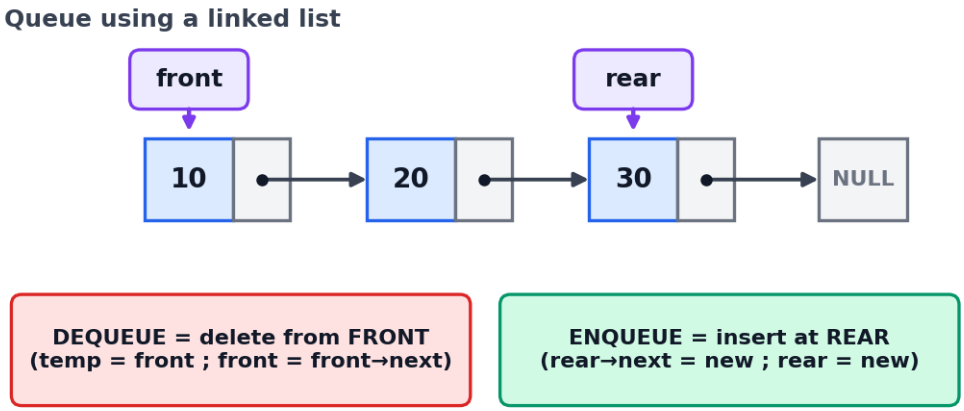


Fig: Linked queue – front and rear pointers

- Each element is a node; **front** points to the first node, **rear** to the last.
- **Enqueue** = insert at the end (O(1) thanks to rear). **Dequeue** = delete the first node (O(1)).
- Dynamic size – no overflow; empty when **front == NULL**.

```
struct Node { int data; struct Node *next; };
struct Node *front = NULL, *rear = NULL;

void enqueue(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));
    n->data = x;  n->next = NULL;
    if (rear == NULL) front = rear = n;     // first node
    else { rear->next = n; rear = n; }      // attach at the end
}
int dequeue() {
    if (front == NULL) { printf("Queue Underflow\n"); return -1; }
    struct Node *t = front;  int x = t->data;
    front = front->next;
    if (front == NULL) rear = NULL;         // queue became empty
    free(t);
    return x;
}
```

Basis	Array queue	Linked queue
Size	Fixed	Dynamic
Overflow	Possible	Practically none
Extra memory	None	One pointer per node
Implementation	Simple, needs circular logic	Pointer handling, free()

COMMON MISTAKE
When the last node is dequeued, set **rear = NULL** as well – otherwise rear keeps pointing to freed memory.

4.4 Priority Queue and Types of Queues

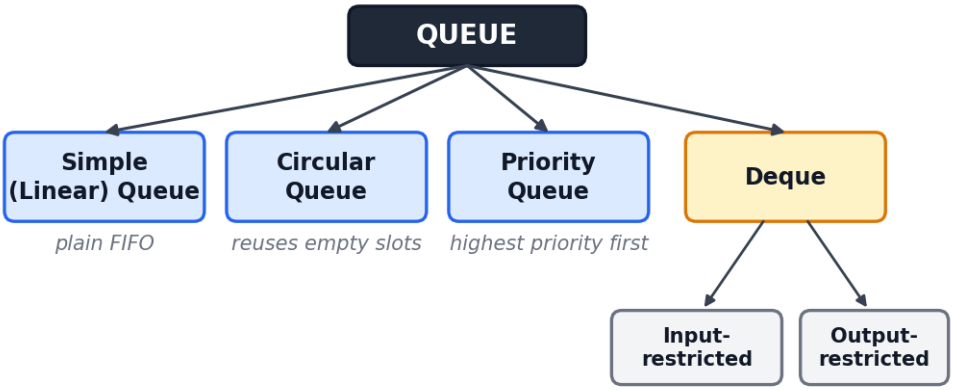


Fig: Family of queues

Type	Rule	Use
Simple queue	FIFO, linear array	Basic waiting lines
Circular queue	FIFO with wrap-around	Buffers, CPU scheduling
Priority queue	Element with **highest priority** leaves first; equal priority → FIFO	OS job scheduling, hospital emergency, Dijkstra
Deque	Insert / delete at both ends	Sliding window, palindromes

Priority queue – key points

- Each element carries a **priority** value; **dequeue** removes the highest (or lowest) priority element, not the oldest.
- Implementation: **unsorted array / list** (insert O(1), delete O(n)), **sorted list** (insert O(n), delete O(1)) or a **heap** (both O(log n)).

Arrival order	Task (priority)	Dequeue order
1	Print report (3)	3rd
2	Fire alarm (1 = highest)	**1st**
3	Backup (4)	4th
4	Email sync (2)	2nd

Priority queue using a sorted linked list

```
struct PNode { int data, pri; struct PNode *next; };
struct PNode *head = NULL;                        // highest priority (smallest number) first

void insertP(int x, int p) {
    struct PNode *n = (struct PNode*) malloc(sizeof(struct PNode));
    n->data = x;  n->pri = p;
    if (head == NULL || p < head->pri) { n->next = head; head = n; return; }
    struct PNode *t = head;
    while (t->next != NULL && t->next->pri <= p) t = t->next;   // FIFO if equal
    n->next = t->next;  t->next = n;
}
int deleteP() {                                   // remove the highest priority element
    struct PNode *t = head;  int x = t->data;
    head = head->next;  free(t);  return x;
}
```

Insertion O(n), deletion O(1)

Queue and stack from each other

Task	Idea
Queue using two stacks	enqueue → push on stack S1. dequeue → if S2 is empty, pop everything from S1 and push into S2 (reverses the order); then pop S2.
Stack using two queues	push → enqueue into Q2, then move all of Q1 behind it; swap names. pop → dequeue from Q1.
Reverse a queue	Dequeue every element into a stack, then pop and enqueue them back.

4.5 Applications of Queues

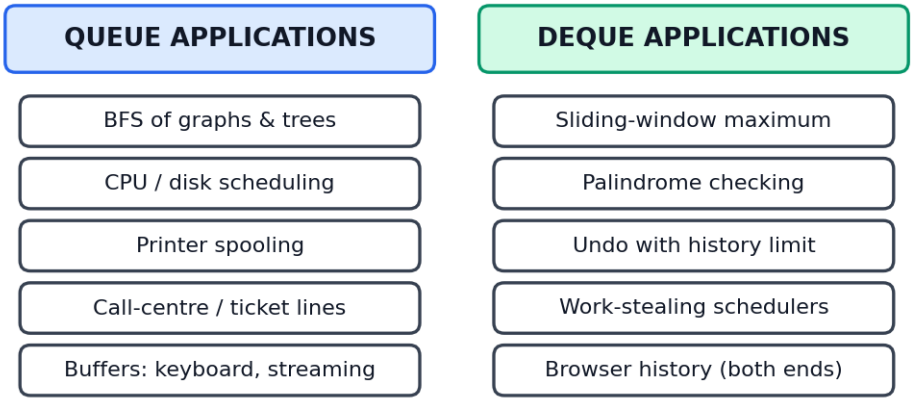


Fig: Typical applications of queues and deques

A. Breadth-First Search (BFS)

- BFS visits a graph **level by level**: start vertex, then all its neighbours, then their neighbours...
- A **queue** keeps the vertices waiting to be explored (FIFO ensures the nearest vertices are served first).
- Uses: **shortest path in an unweighted graph**, social-network "friends of friends", web crawling, level-order traversal of trees.

Algorithm

1. Mark the start vertex visited and **enqueue** it.
2. While the queue is not empty: **dequeue** a vertex v and visit (print) it.
3. For every unvisited neighbour w of v: mark visited and **enqueue** w.

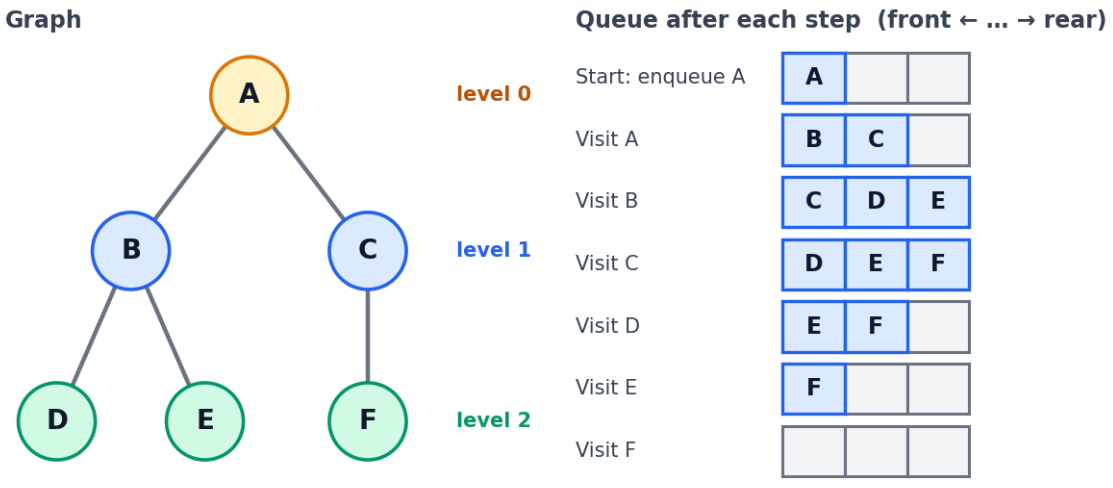


Fig: BFS from A – order of visiting: A → B → C → D → E → F

```
void BFS(int g[N][N], int start) {                // adjacency matrix g
    int visited[N] = {0};
    enqueue(start);  visited[start] = 1;
    while (!isEmpty()) {
        int v = dequeue();
        printf("%d ", v);
        for (int w = 0; w < N; w++)
            if (g[v][w] && !visited[w]) { visited[w] = 1; enqueue(w); }
    }
}
```

Time O(V + E) with adjacency lists, O(V²) with an adjacency matrix

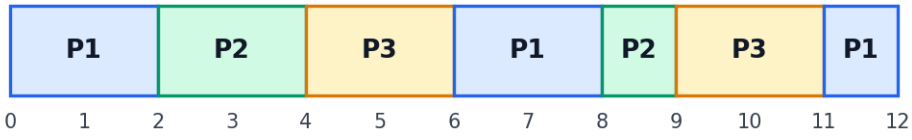
REMEMBER

BFS → **Queue** (level by level). DFS → **Stack** (go deep, then backtrack).

B. CPU scheduling (round robin)

- Ready processes wait in a **circular queue**.
- The CPU gives each process a fixed **time quantum**; if unfinished, it goes to the **rear** of the queue.
- First-Come-First-Served (FCFS) scheduling is a plain FIFO queue.

Round-robin scheduling, time quantum = 2 (P1 = 5, P2 = 3, P3 = 4 units)



Time units →

The ready queue rotates: P1 → P2 → P3 → P1 → … (finished processes leave the queue)

Fig: Gantt chart of round-robin scheduling

Process	Burst time	Finish time	Turnaround (finish − arrival 0)	Waiting (turnaround − burst)
P1	5	12	12	7
P2	3	9	9	6
P3	4	11	11	7

Ready-queue snapshots for the chart above (front on the left):

Time	CPU runs	Ready queue after the slice
0 – 2	P1 (3 left)	P2, P3, P1
2 – 4	P2 (1 left)	P3, P1, P2
4 – 6	P3 (2 left)	P1, P2, P3
6 – 8	P1 (1 left)	P2, P3, P1
8 – 9	P2 (done)	P3, P1
9 – 11	P3 (done)	P1
11 – 12	P1 (done)	empty

C. Other applications

- **Printer spooler** and **disk scheduling** – requests served in arrival order.
- **Buffers** – keyboard input, video streaming, data between fast and slow devices.
- **Call centres, ticket booking** – customers served in order.
- **Level-order traversal** of trees (Unit V).
- **Simulation** of real-world waiting lines (banks, traffic).

4.6 Deques (Double-Ended Queues)

DEFINITION
Deque (pronounced “deck”) = a linear structure that allows **insertion and deletion at both ends** – front and rear.

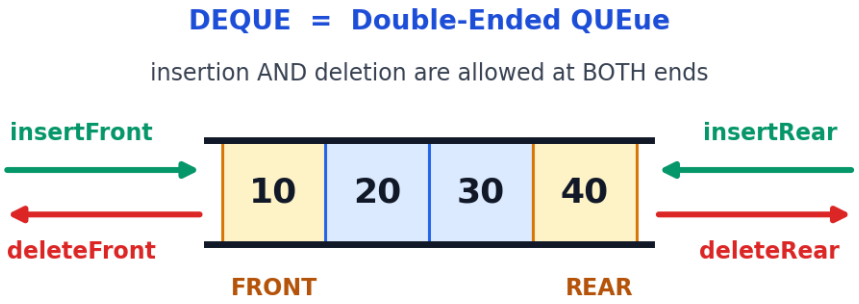


Fig: A deque supports four basic operations

Operation	Action	Time
insertFront(x)	Add at the front	O(1)
insertRear(x)	Add at the rear	O(1)
deleteFront()	Remove from the front	O(1)
deleteRear()	Remove from the rear	O(1)
getFront() / getRear()	Read either end	O(1)

Types of deque

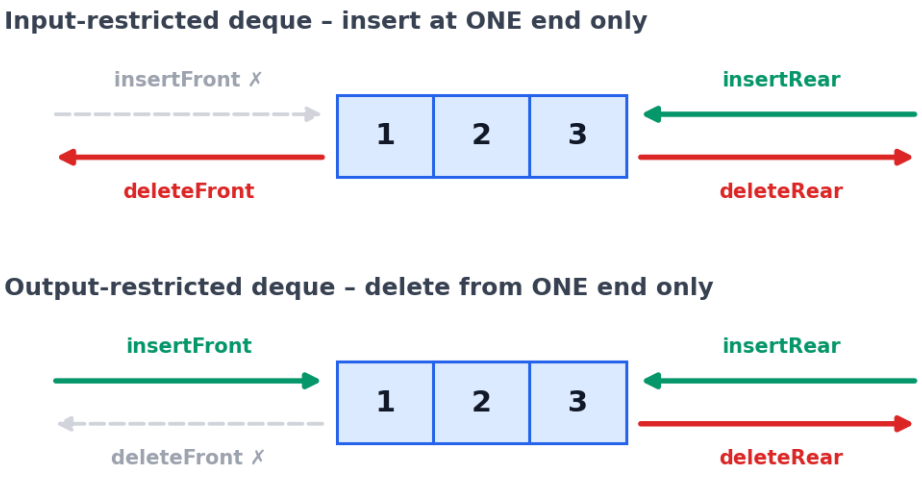


Fig: Input-restricted and output-restricted deques

- **Input-restricted:** insertion at **one end only**, deletion at **both** ends.
- **Output-restricted:** deletion at **one end only**, insertion at **both** ends.
- A deque used from **one end only** behaves like a **stack**; used with insert-rear / delete-front it behaves like a **queue**.

Implementation

- **Circular array:** front and rear wrap around; insertFront uses **front = (front − 1 + MAX) % MAX**.
- **Doubly linked list:** prev and next pointers make all four operations O(1) with no size limit.

```
int dq[MAX], front = -1, rear = -1;                    // circular array deque

void insertFront(int x) {
    if ((rear + 1) % MAX == front) { printf("Full\n"); return; }
    if (front == -1) front = rear = 0;
    else front = (front - 1 + MAX) % MAX;              // move backwards with wrap
    dq[front] = x;
}
void insertRear(int x) {
    if ((rear + 1) % MAX == front) { printf("Full\n"); return; }
    if (front == -1) front = rear = 0;
    else rear = (rear + 1) % MAX;
    dq[rear] = x;
}
int deleteFront() {
    if (front == -1) { printf("Empty\n"); return -1; }
    int x = dq[front];
    if (front == rear) front = rear = -1; else front = (front + 1) % MAX;
    return x;
}
int deleteRear() {
    if (front == -1) { printf("Empty\n"); return -1; }
    int x = dq[rear];
    if (front == rear) front = rear = -1; else rear = (rear - 1 + MAX) % MAX;
    return x;
}
```

Palindrome check using a deque (example: “MADAM”)

Step	Deque	Compare front and rear	Result
1	M A D A M	M = M	delete both ends
2	A D A	A = A	delete both ends
3	D	only one element left	**Palindrome**

Applications of deques

- **Palindrome check** – compare the front and rear characters, delete both, repeat.
- **Sliding-window maximum / minimum** problems.
- **Undo / redo with limited history** – oldest action is dropped from one end, newest added at the other.
- **Work stealing** in multi-core schedulers; **A-Steal** task queues.
- Browser history and **both-direction** traversal of items.

4.7 Quick Revision & Practice

ONE-PAGE SUMMARY

- Queue = **FIFO**; insert at **rear**, delete at **front**; all operations O(1).
- Linear array queue wastes freed slots → use **circular queue** with **(i+1) % MAX**.
- Linked queue: front for delete, rear for insert; empty when front == NULL.
- Priority queue: highest priority leaves first.
- BFS uses a **queue**; round-robin scheduling uses a **circular queue**.
- Deque = both ends; input-restricted / output-restricted variants.

Likely exam questions

2 marks

- Define queue. State its principle.
- Differentiate stack and queue.
- What is a circular queue? Why is it needed?
- What is a deque? Name its two types.

5 marks

- Write algorithms for enqueue and dequeue in a circular queue.
- Explain queue implementation using a linked list.
- Explain BFS with an example and trace the queue.
- Explain priority queue with an example.

10 marks

- Explain the queue ADT with array and linked-list implementations and applications.
- Explain deque operations, types and implementation with diagrams.

Try it yourself

1. In a circular queue of size 5, perform: enq 10, 20, 30, 40; deq, deq; enq 50, 60, 70. Show front and rear after each step.
2. Run BFS on a graph with edges 1-2, 1-3, 2-4, 3-4, 4-5 from vertex 1. *(Answer: 1 2 3 4 5)*
3. Perform round robin (quantum 3) for P1 = 7, P2 = 4, P3 = 5 and draw the Gantt chart.
4. Reverse the first k elements of a queue using a stack.